



Sichuan University -
Pittsburgh Institute

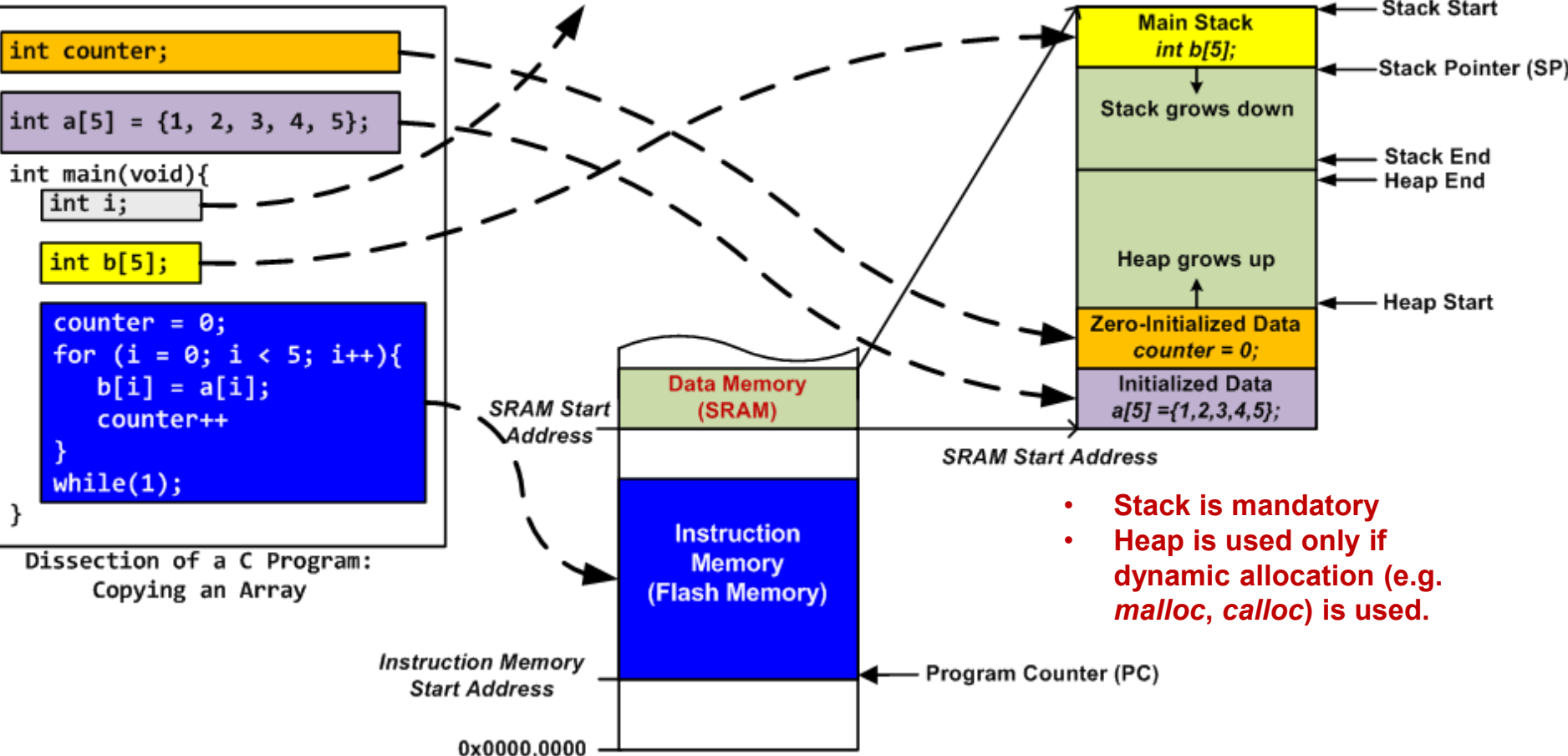
ECE0202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute



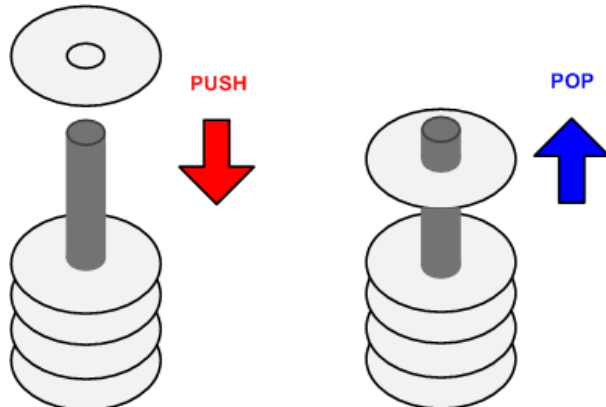
Loading Code and Data into Memory

To improve performance, some variables are not stored in memory. Variable *i* will be stored in a register.

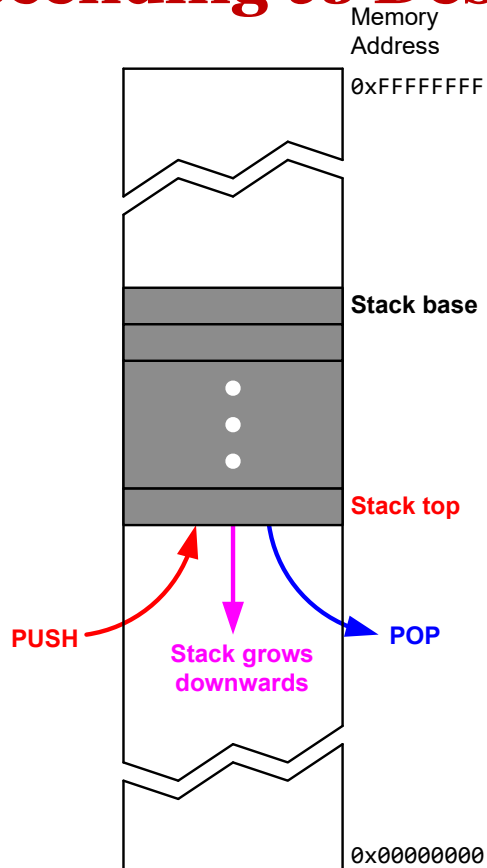


Stack

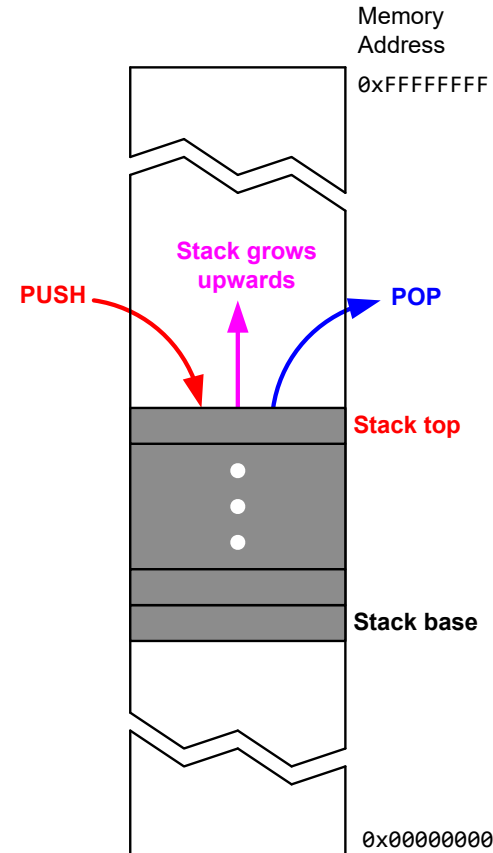
- A **Last-In-First-Out** data structure
- Only allow to access the most recently added item
 - Also called the top of the stack
- Key operations:
 - push (add item to stack)
 - pop (remove top item from stack)



Stack Growth Convention: *Ascending vs Descending*

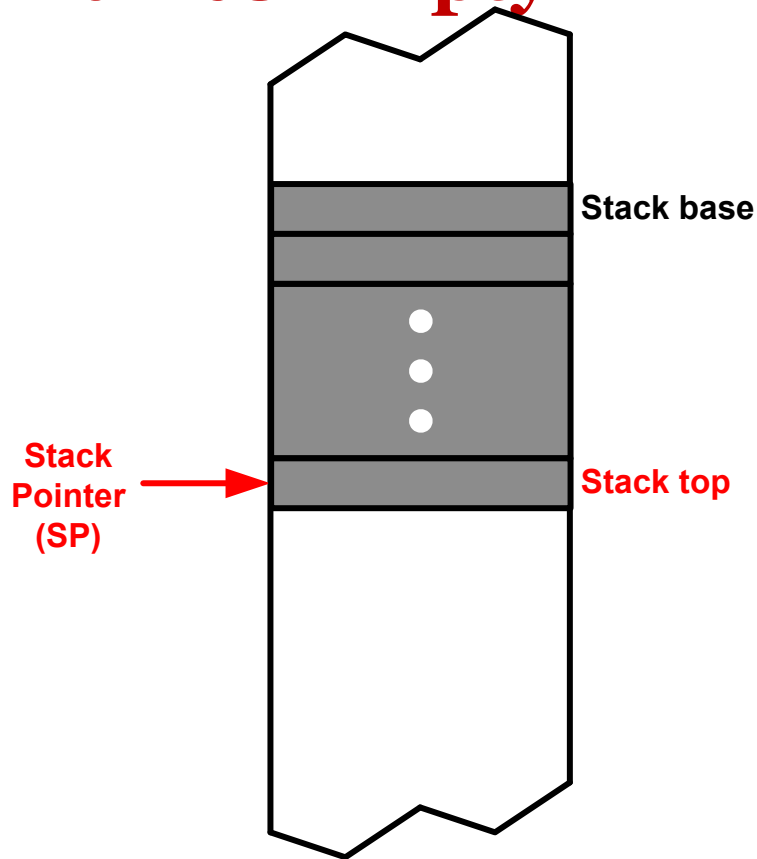


Descending stack: Stack grows towards low memory address

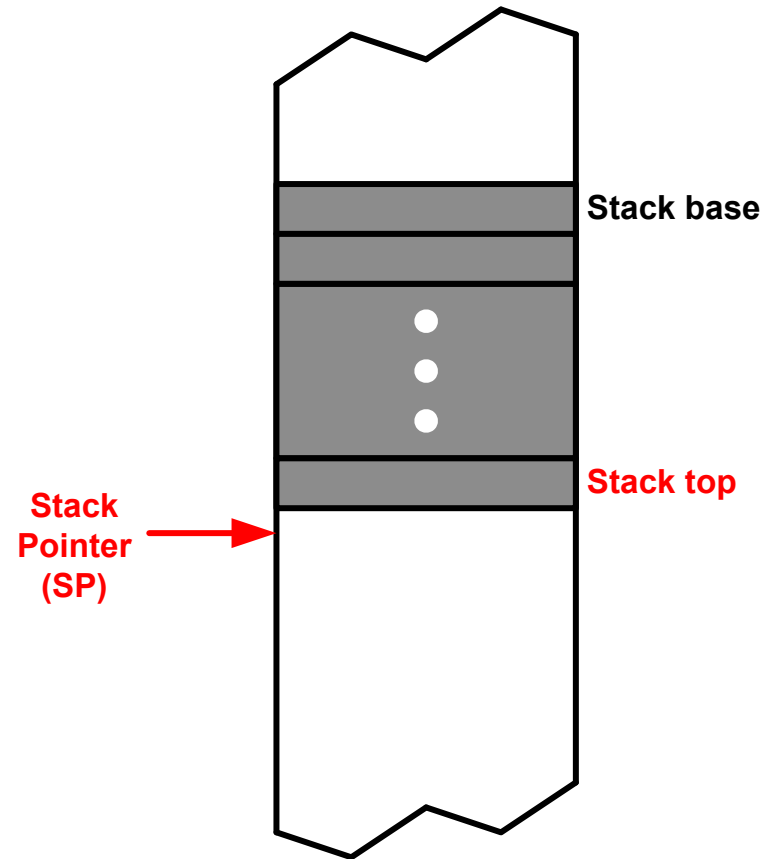


Ascending stack: Stack grows towards high memory address

Stack Growth Convention: **Full vs Empty**



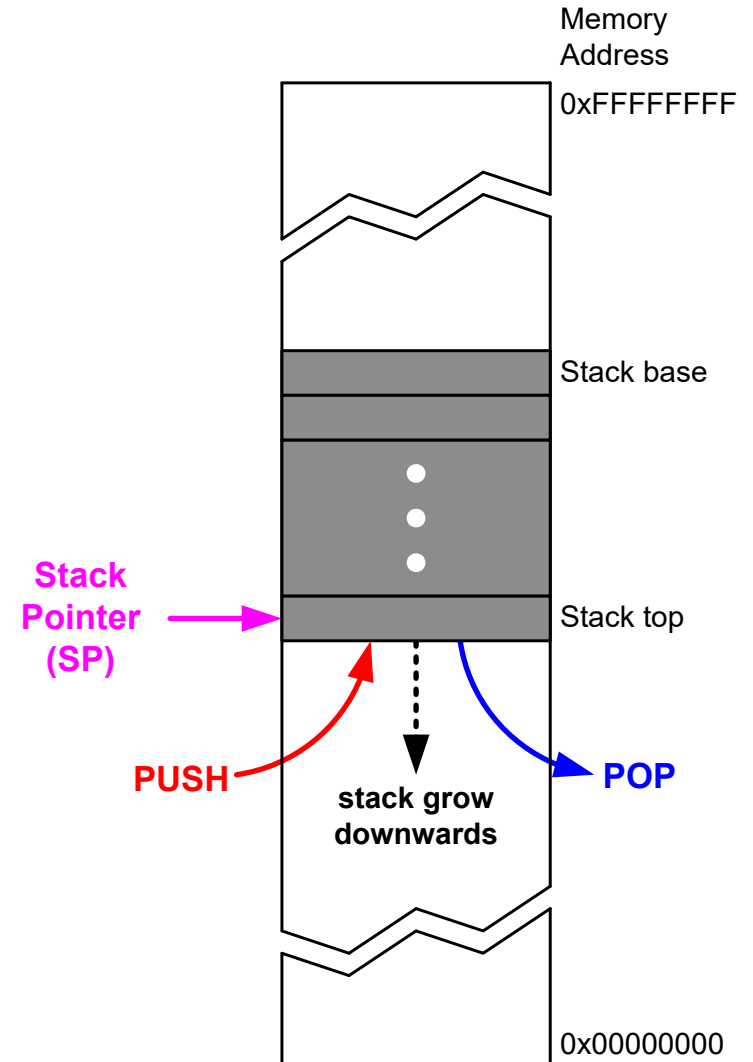
Full stack: SP points to the last item pushed onto the stack



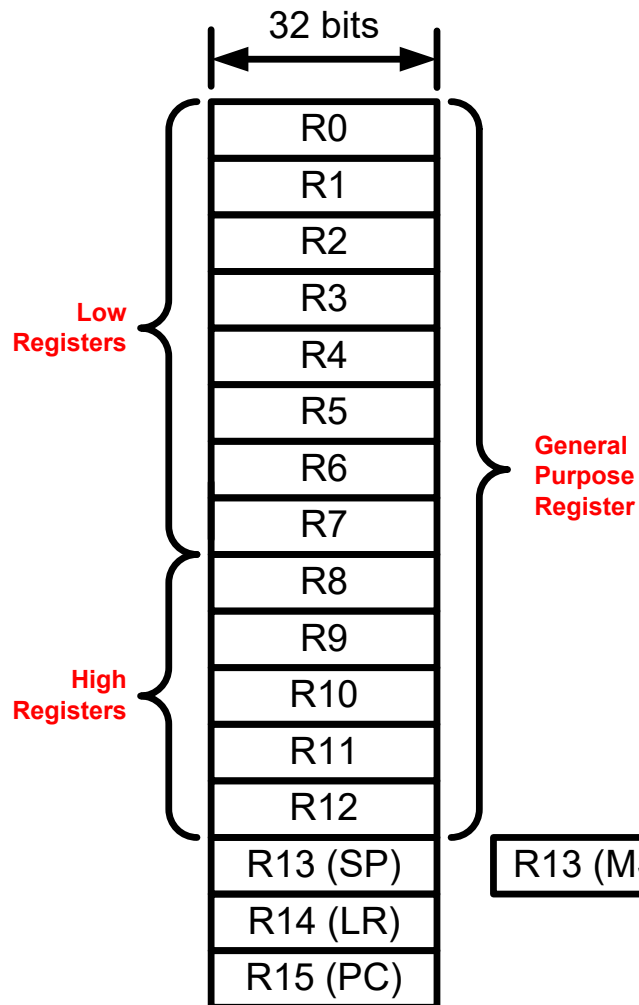
Empty stack: SP points to the next free space on the stack

Cortex-M Stack

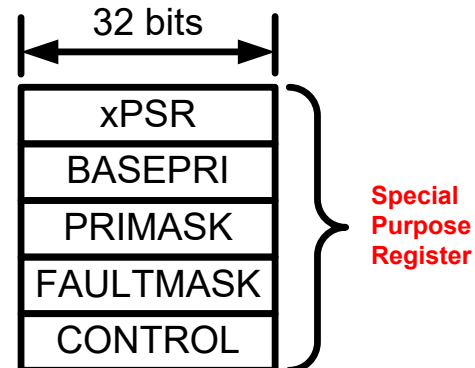
- stack pointer (SP) = R13
- Cortex-M uses *full descending stack*
- stack pointer
 - decremented on **PUSH**
 - incremented on **POP**



Processor Registers



- ▶ Fastest way to read and write
- ▶ Registers are within the processor chip
- ▶ A register stores 32-bit value
- ▶ STM32L has
 - ▶ **R0-R12**: 13 general-purpose registers
 - ▶ **R13**: Stack pointer (Shadow of MSP or PSP)
 - ▶ **R14**: Link register (LR)
 - ▶ **R15**: Program counter (PC)
 - ▶ Special registers (xPSR, BASEPRI, PRIMASK, etc)



Initializing the stack pointer (SP)

- Before using the stack, software has to define stack space and initialize the stack pointer (SP).
- The assembly file **startup.s** defines stack space and initialize SP.
- SP starts at **0x20000200** for STM32

Stack

PUSH {Rd}

- $SP = SP - 4 \rightarrow$ descending stack
- $(*SP) = Rd \rightarrow$ full stack

Push multiple registers

They are equivalent.

PUSH {r6, r7, r8} $\longleftrightarrow$ **PUSH {r8, r7, r6}** $\longleftrightarrow$ **PUSH {r8}**
PUSH {r7}
PUSH {r6}

- The order in which registers listed in the register list does not matter.
- When pushing multiple registers, these registers are automatically **sorted by name** and **the lowest-numbered register** is stored to the lowest memory address, *i.e.* **is stored last**.

Stack

POP {Rd}

- $Rd = (*SP) \rightarrow$ full stack
- $SP = SP + 4 \rightarrow$ Stack shrinks

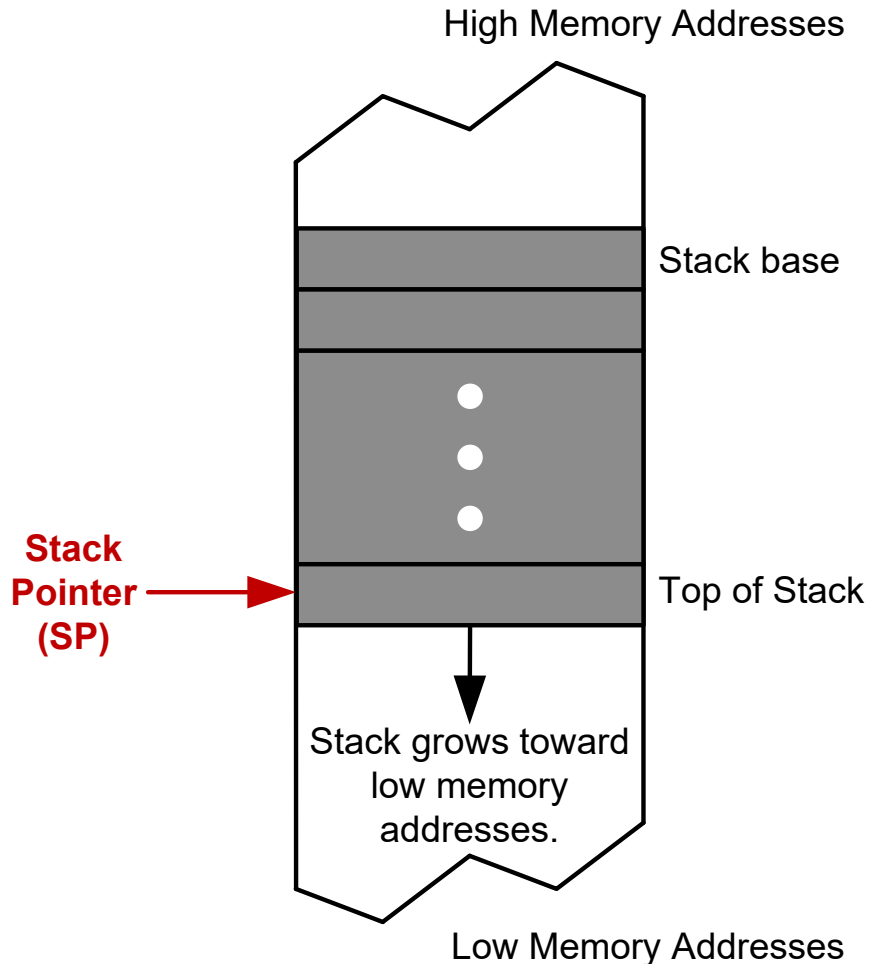
Pop multiple registers

They are equivalent.

POP {r6, r7, r8} $\longleftrightarrow$ POP {r8, r7, r6} $\longleftrightarrow$ POP {r6}
POP {r7}
POP {r8}

- The order in which registers listed in the register list does not matter.
- When popping multiple registers, these registers are automatically **sorted by name** and **the lowest-numbered register** is loaded from the lowest memory address, *i.e.* **is loaded first**.

Full Descending Stack



PUSH {register_list}
equivalent to:
STMDB SP!, {register_list}

DB: Decrement Before

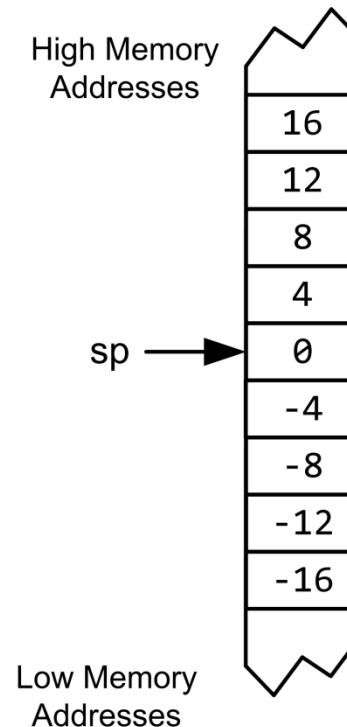
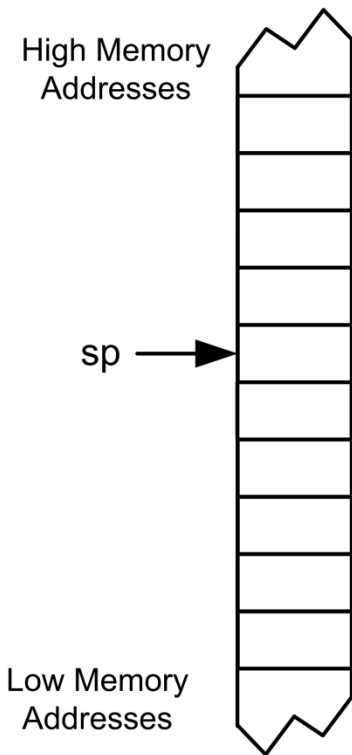
POP {register_list}
equivalent to:
LDMIA SP!, {register_list}

IA: Increment After

Full Descending Stack

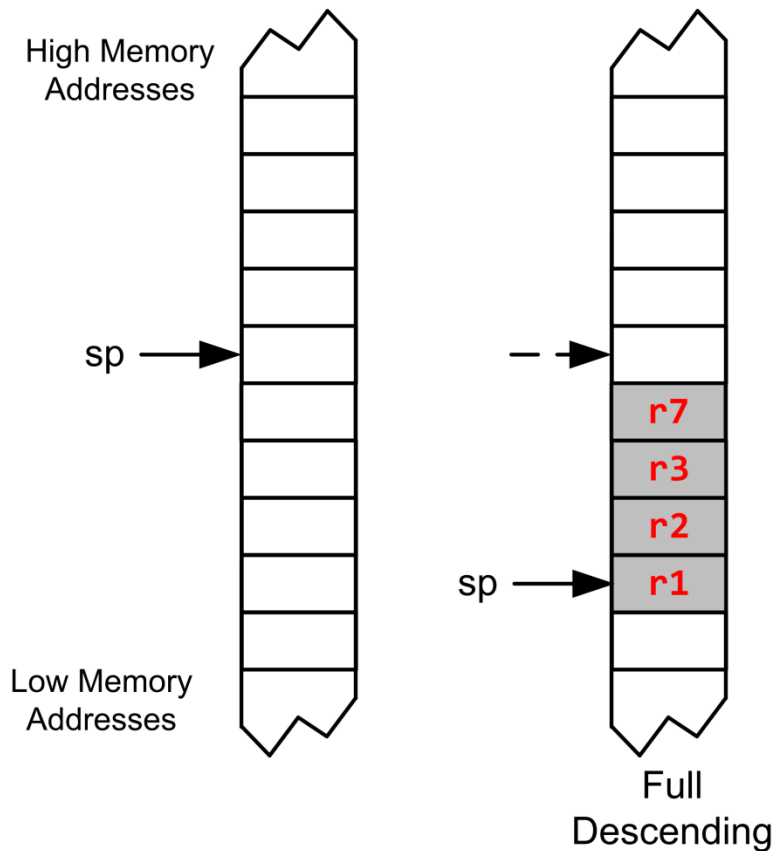
PUSH {r3, r1, r7, r2}

POP {r3, r1, r7, r2}

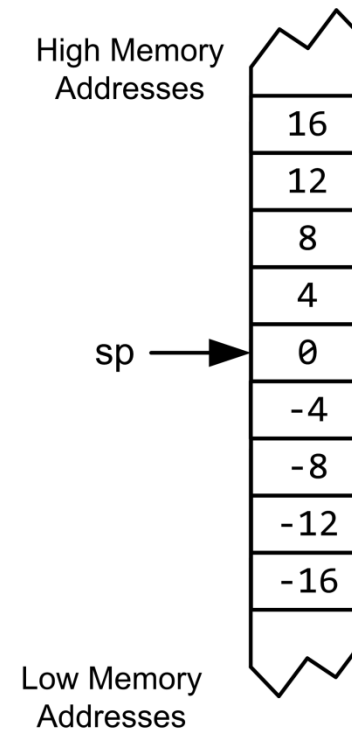


Stack

PUSH {r3, r1, r7, r2}

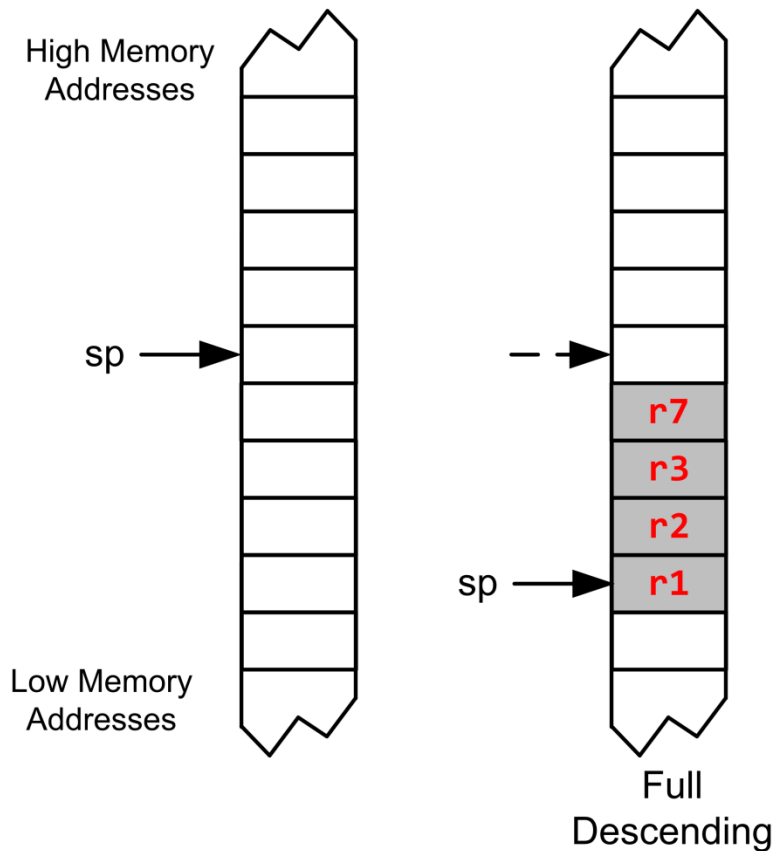


POP {r3, r1, r7, r2}

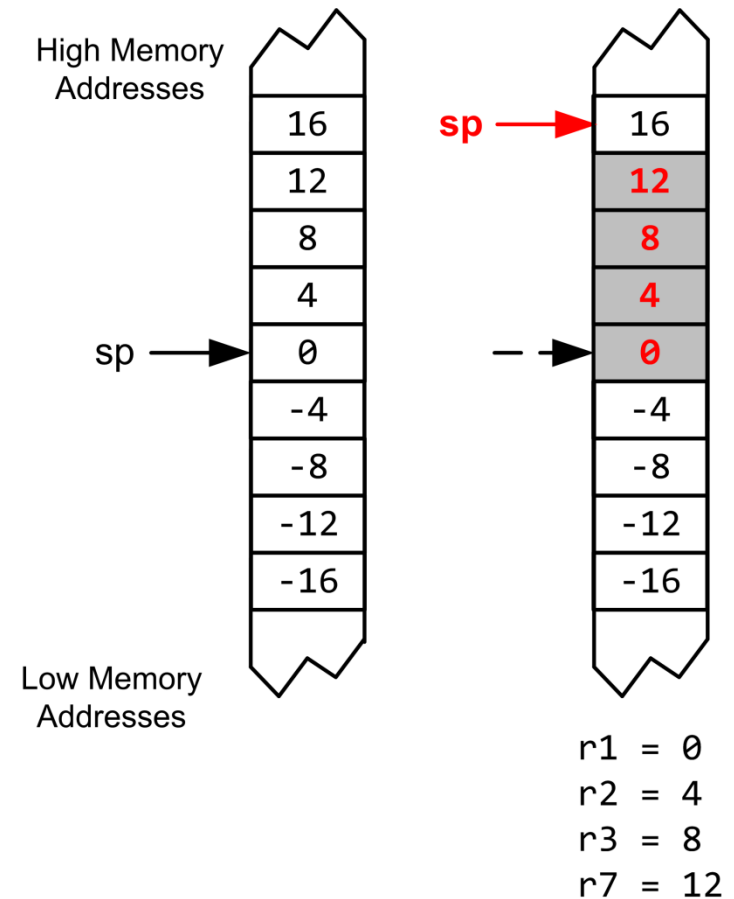


Stack

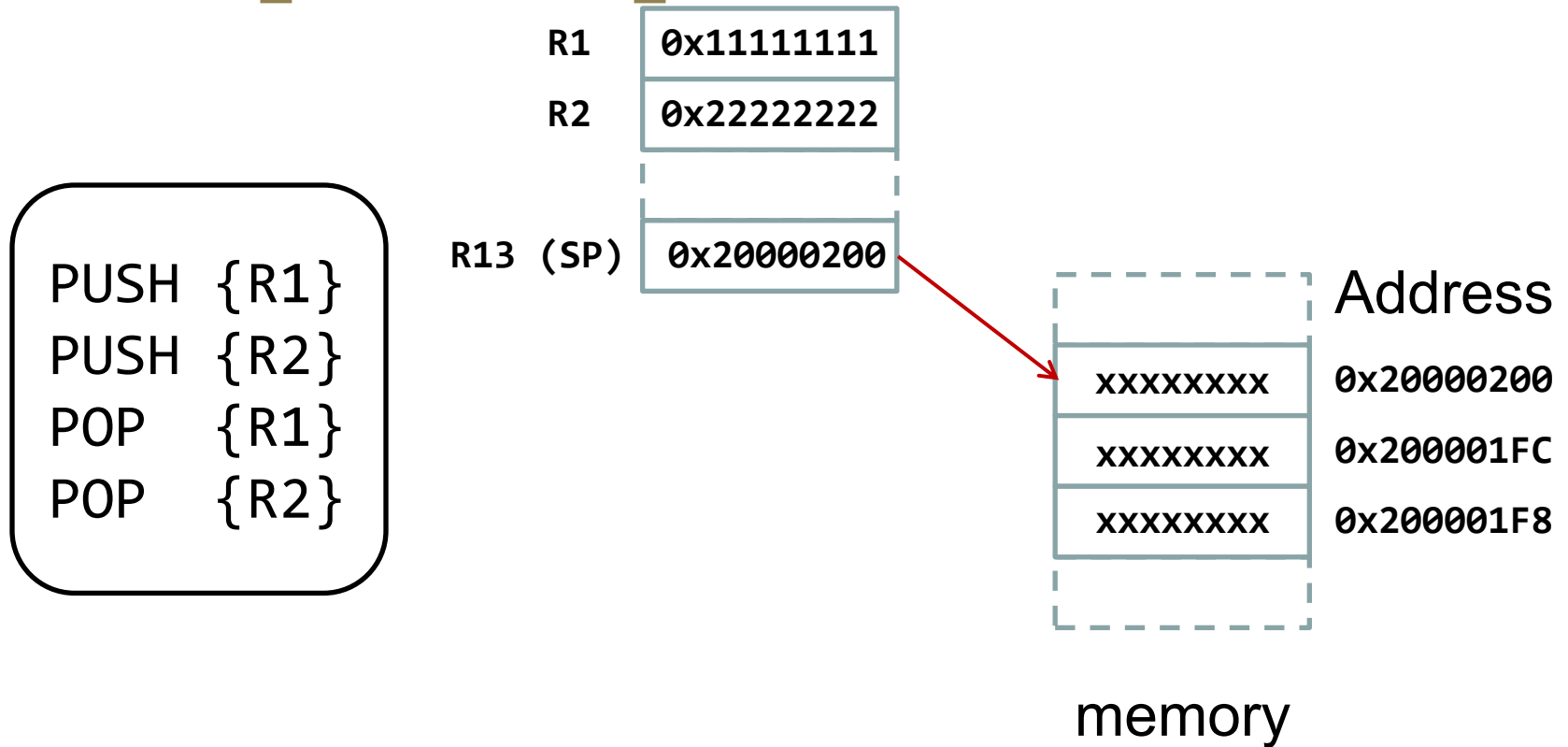
PUSH {r3, r1, r7, r2}



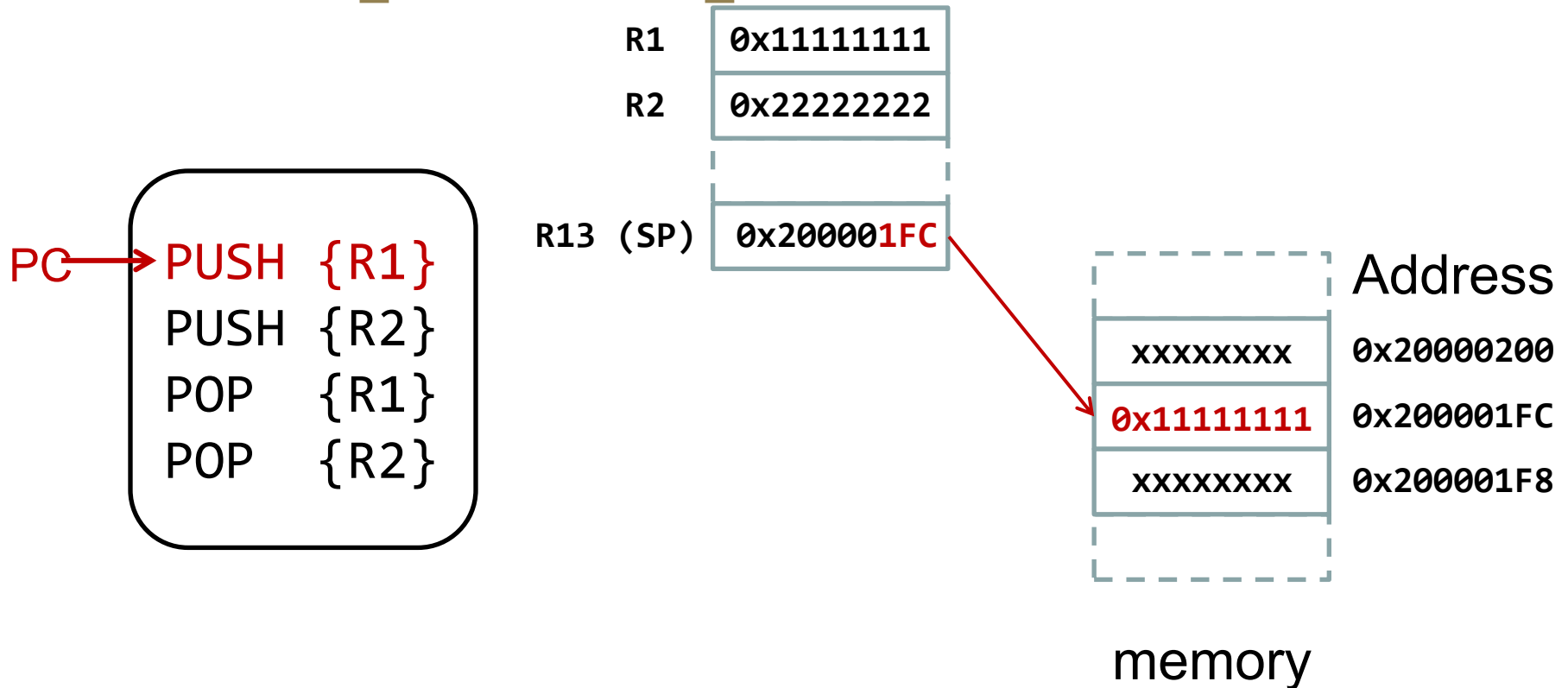
POP {r3, r1, r7, r2}



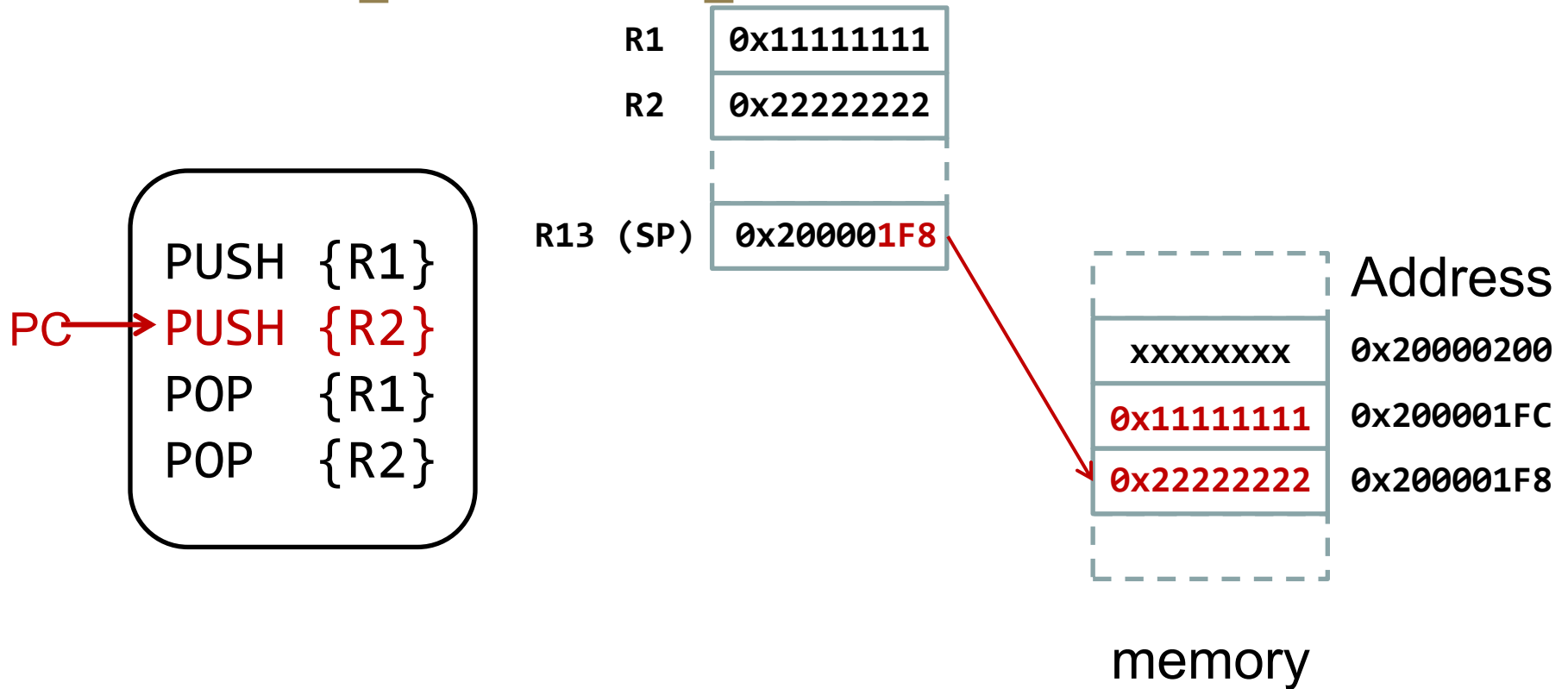
Example: swap R1 & R2



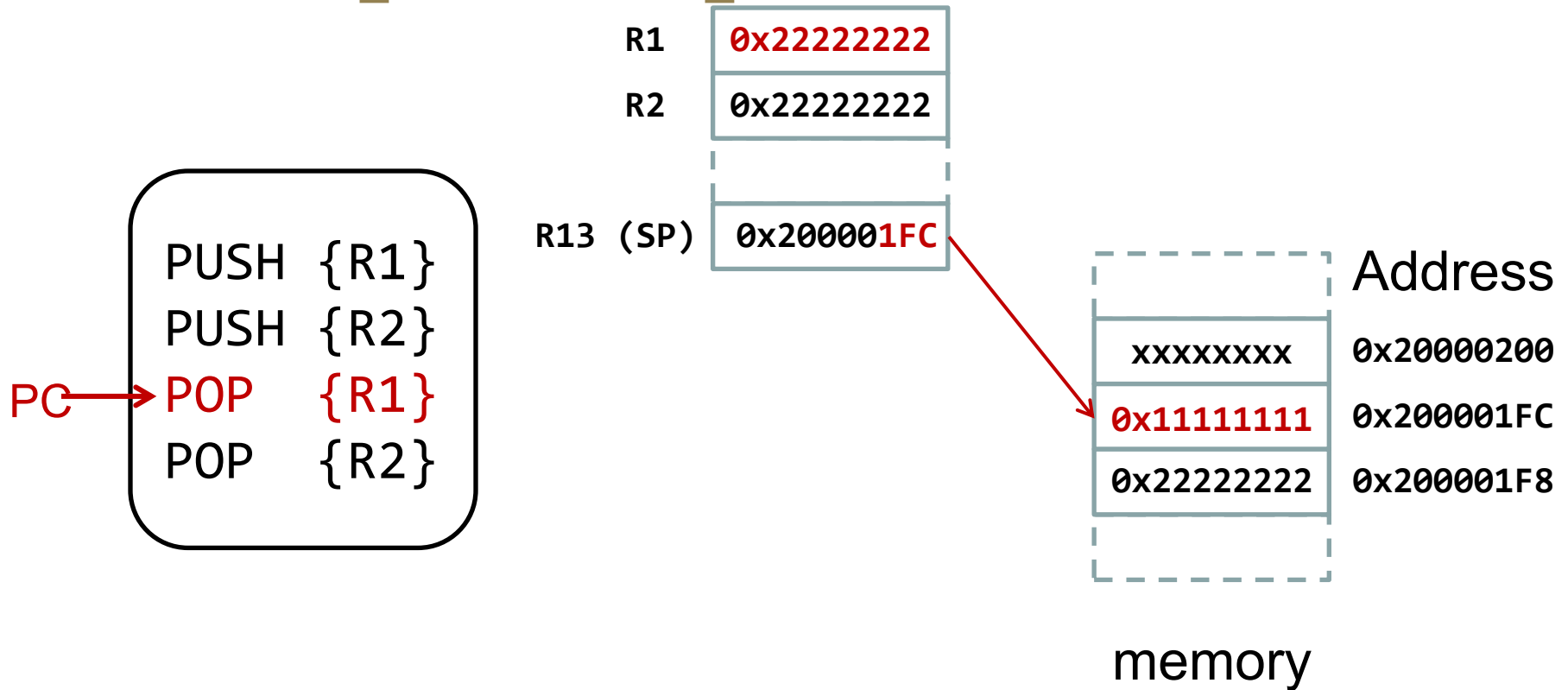
Example: swap R1 & R2



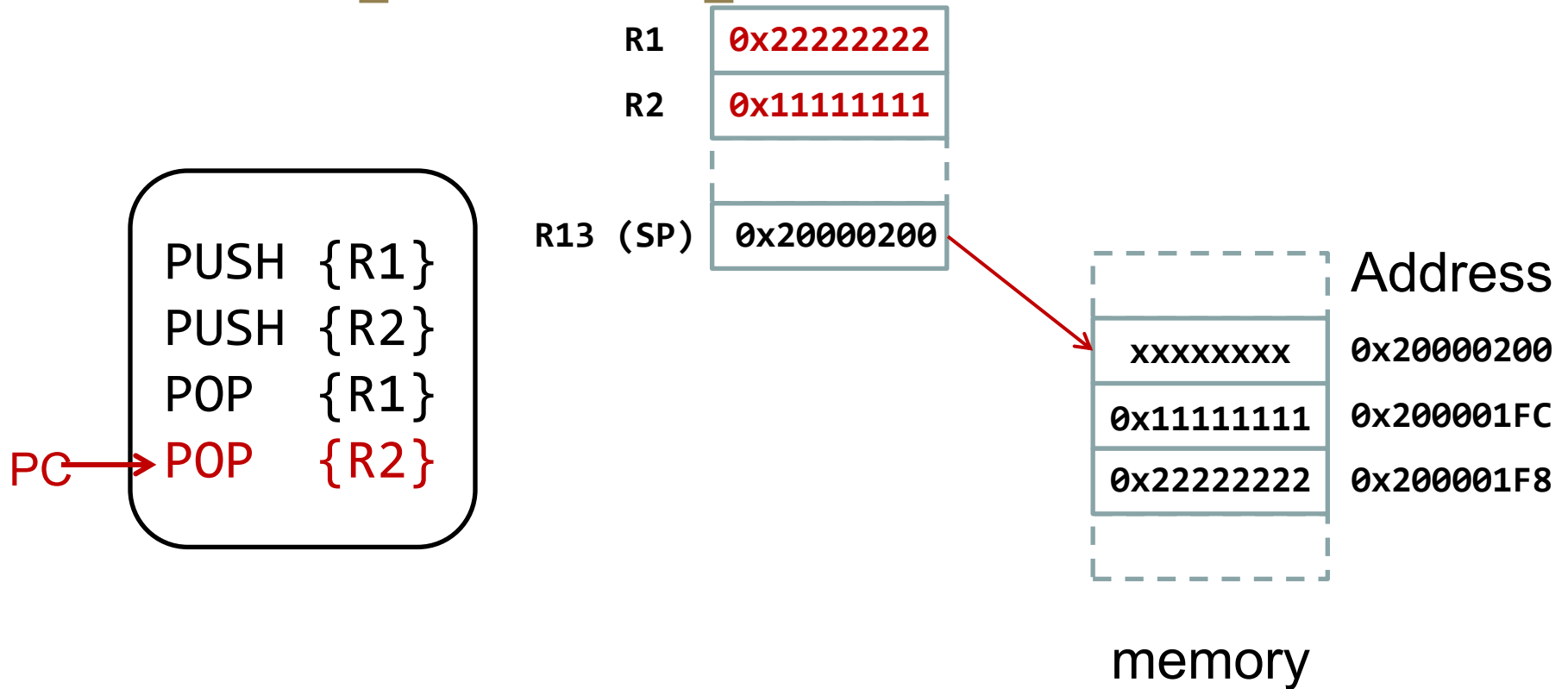
Example: swap R1 & R2



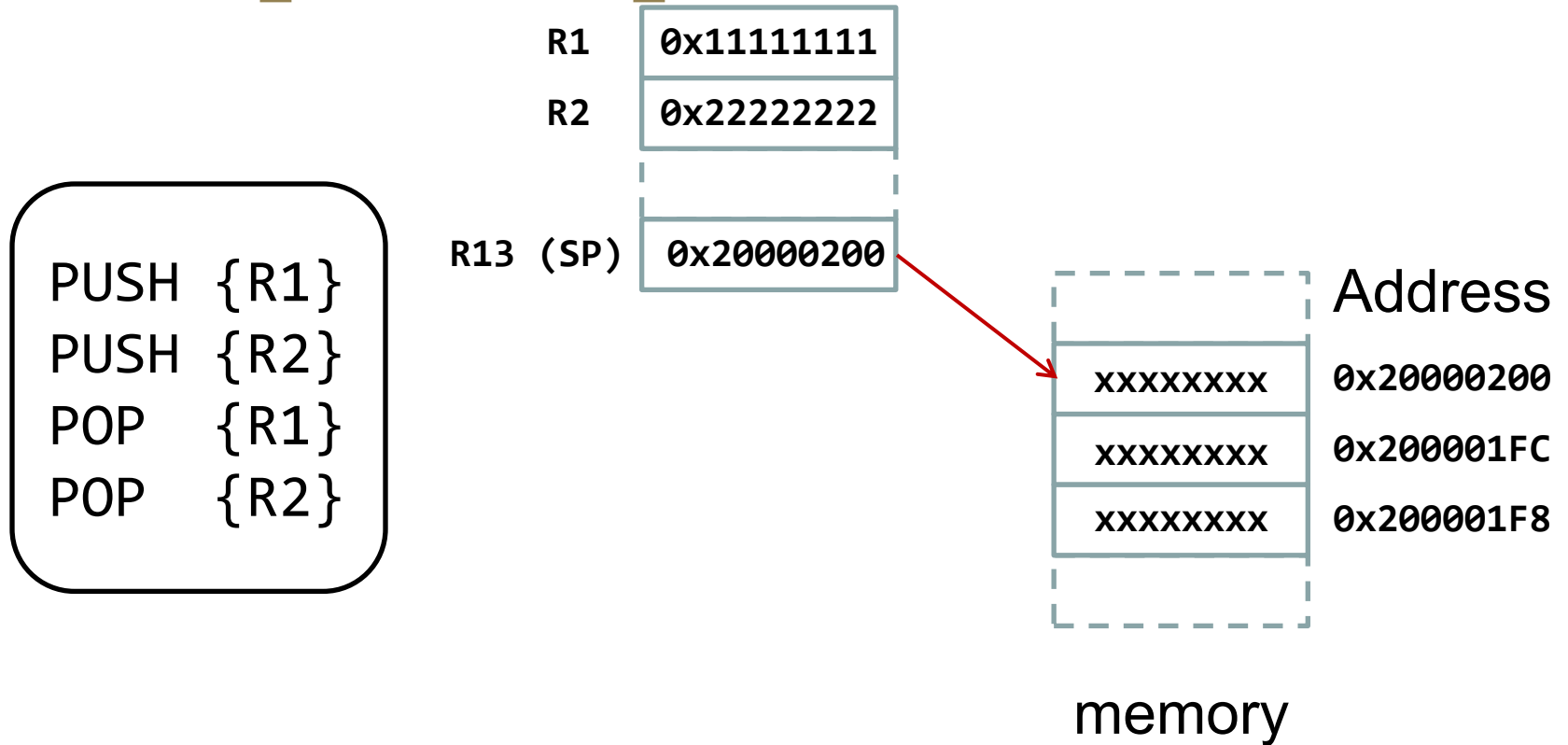
Example: swap R1 & R2



Example: swap R1 & R2



Example: swap R1 & R2



```
PUSH {R1}
PUSH {R2}
POP {R1, R2}
```

```
PUSH {R1, R2}
POP {R1}
POP {R2}
```